SEPARATE PARAMETRIC PINN STUDY · 04 NOVEMBER 2026

Train once, reuse across random networks

Parametric graph PINN versus the verified sparse solver. A neural operator receives the full random conductance field and physical parameters, predicts hydraulic head, and estimates effective conductance. Trained weights are reused without a new sparse solve or per-input weight fitting. Every forward evaluation still performs K fixed iterative updates of head.

> On 9,000 independent held-out scenario cases (4,518 with positive flow), the K=2048 model has mean absolute relative conductance error **0.0200%**, 95th-percentile error **0.0393%**, and maximum error **4.7386%**. The zero-flow decision is exact graph preprocessing, not a learned classifier.

Property	Direct sparse method	Parametric graph PINN
Equations	Kirchhoff conservation	Same graph physics; energy loss
Inputs	Full edge conductance field, heads	Full field + δ,p,σ + heads
Offline fitting	None	4,096 spanning training fields; no solution labels
New input	New sparse solve	Graph preprocessing + fixed K neural layers
Effective conductance	Inlet flux / Δh	Energy-derived upper-bound estimate
Conservation	Linear-solver accuracy	Approximate; tested separately
Boundary / disconnected graph	Exact	Exact structural constraints

Practical conclusion. The reusable model works accurately for most in-domain conductance predictions, but errors are not uniformly small. It is an unrolled learned iterative architecture with explicit graph preprocessing, not a closed-form or a single dense-MLP evaluation. The fastest measured PINN setting is 4.35× the sparse G-only latency; no positive break-even is observed. The sparse method remains the preferred precision reference for this inexpensive 25×25 problem.

Why physical scalars alone are insufficient

Two networks with identical δ,p,σ can have different topology and conductance fields, including one spanning and one disconnected. In 1000 test cases with identical δ=0, p=0.5 and σ=1, standard deviation of G was 0.02097; even the best empirical constant predictor had RMSE 0.02096. The model therefore takes the full field, or a seed identifying it. With only δ and σ, the API reports ensemble statistics from sampled fields, not a unique realization.

1 · Physics, parameters and architecture

Use the same n=25 lattice, 625 nodes and 1,200 possible edges. For each realization p=0.5+δU, U~Uniform(−1,1); active edges have g=exp(σZ). The neural conditioning scalars include δ,p,σ; node inputs include coordinates, conductance and topology descriptors, and shortest resistance distances to the two boundaries. Given the full g field, δ,p,σ are redundant distribution descriptors, not additional forces in Kirchhoff's law.

$$E(h;g) = \Sigma_e\, g_e(h_u - h_v)^2,\ h_{left}=1,\ h_{right}=0$$
$$\partial E/\partial h_i = 2\,\Sigma_j g_{ij}(h_i - h_j) = 0$$
$$G_{exact} = \min_h E(h;g),\ G_{est} = E(h_\theta;g) \geq G_{exact}$$

The inequality holds for admissible fixed-boundary heads in exact arithmetic; it does not certify a tight error bound. Away from equilibrium, E(hθ) and the inlet flux from hθ need not agree, so both are reported separately.

A discrete, parametric PINN

1. Connected components impose the exact head on one-boundary clusters and identify zero-flow networks. Floating clusters carry zero flow and have undefined absolute head.
2. Dijkstra distances use resistances 1/g. Initialize h_0=d_right/(d_left+d_right) on spanning components. Uniform conductance scaling is canonicalized by the active-link median.
3. A 21→32→32→1 node encoder predicts a bounded initial correction. A 6→16→2 parametric MLP predicts graph-level α,β. There are 1,939 trainable parameters.
4. Apply a fixed number K of learned Kirchhoff-residual message-passing layers; clamp final heads to [0,1], which cannot increase graph energy.

$$P_{ij} = g_{ij} / \Sigma_j g_{ij}$$
$$h^{k+1} = h^k - \alpha(h^k - Ph^k) + \beta(h^k - h^{k-1}),\ \text{on free nodes}$$
$$\beta = 0.999\ \text{sigmoid}(b),\ \alpha = (1+\beta)\ 0.999\ \text{sigmoid}(a)$$

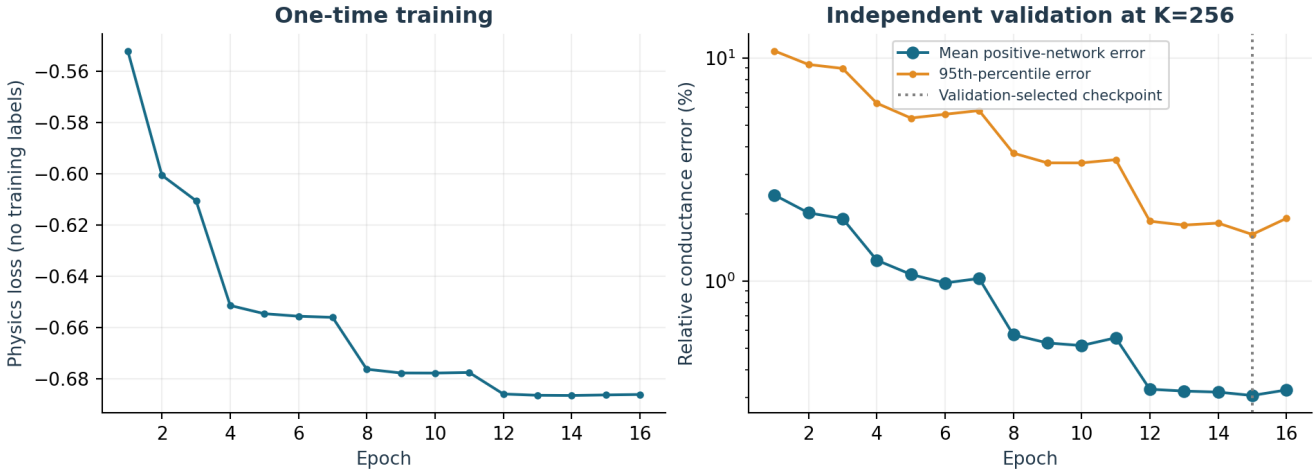
This is a physics-informed unrolled graph network related to variational PINN/Deep Ritz ideas [1,2]. It deliberately contains repeated graph operations. It is neither a continuum coordinate-derivative PINN nor a claim that all computation disappears after training. For arbitrary boundary heads, h=h_R+(h_L−h_R)h_unit and Q_est=G_est(h_L−h_R); scaling all g by c scales G by c exactly.

2 · One-time training and model selection

Train on 4,096 spanning networks, retained from 8,155 independent generated fields. Nonspanning cases need no optimization because their response is fixed structurally. Half of δ and σ draws use scenario values {0,0.1,0.25} and {0,1,2}, respectively; the other half use Uniform(0,0.25) and Uniform(0,2). A single global p is then sampled per network.

$$\text{Loss}(\theta) = \text{mean}\ \log[E(h\theta;g) / E(h_0;g)]$$
$$+\ 0.01\ \text{mean}\ [r^2],\ r = h\theta - Ph\theta \text{ on free nodes and zero elsewhere}$$

The logarithm is monotone; each connected network's energy minimum is the Kirchhoff solution. Training uses **no reference head or conductance labels**. Sparse reference solutions are generated only for 512 independent validation networks and for later paired tests. Adam uses initial learning rate 0.0015, weight decay 10^{-7} and gradient clipping 1. Batch size 32; 16 epochs with depth curriculum 32→64→128→256. Training uses float32; evaluation uses float64, on CPU with two PyTorch threads.



Training physics loss and validation errors. Best epoch is selected by validation mean relative conductance error at K=256.

Training-loop time was 297.4 s; total offline setup, validation and depth selection took 334.7 s. Epoch 15 was selected. Fast mode uses K=256; accurate mode uses K=2048. Larger depth reuses the same learned weights; it is additional fixed message passing, not additional training.

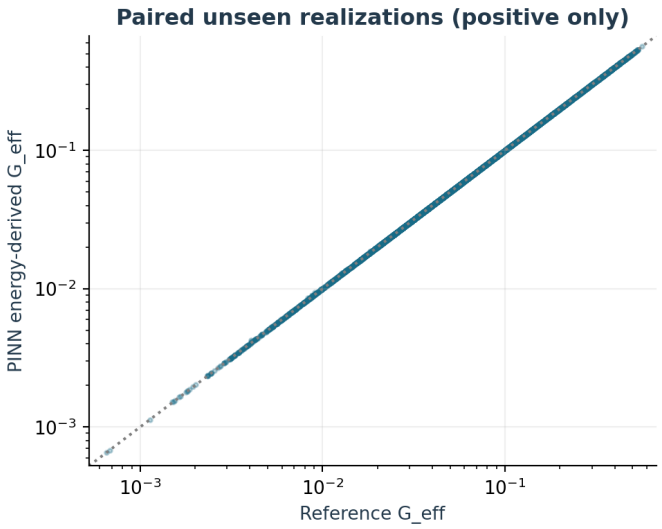
> **Unmet validation target.** The desired validation 99th-percentile G error below 0.1% was not met within the tested depth cap 2048 (observed 0.3707%). The most accurate tested depth was retained, and the failure is not hidden as a guarantee. No settings were tuned to improve the subsequent test results.

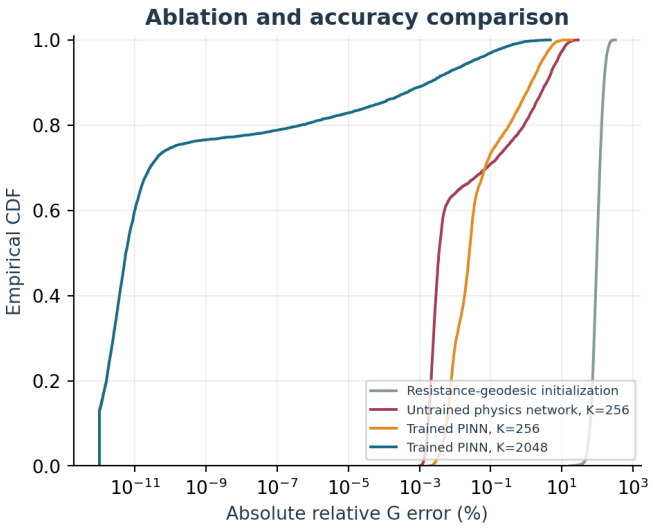
Base SeedSequence 20261105; training stream 101, validation 102, test 201–205, sensitivity 301, bootstrap 302, timing 401. Input fingerprints and exact versions are saved. One training seed was run; training-to-training variation is not quantified.

3 · Paired pointwise accuracy and ablation

Freeze weights and depth settings before testing. The main paired test has nine δ/σ scenarios × 1,000 realizations, plus 1,000 continuous in-domain cases. Each reference and model sees the **same g array**. Relative errors exclude exactly zero reference values; disconnected cases are assessed separately.

Method	Mean |relative G error|	95th pct.	Maximum	Mean free-head RMSE
Geodesic only	103.6598%	174.1826%	319.4120%	0.096001
Untrained, K=256	1.1381%	7.6419%	28.8138%	0.015183
Trained, K=256	0.4705%	2.9005%	18.8355%	0.009909
Trained, K=2048	0.0200%	0.0393%	4.7386%	0.001460



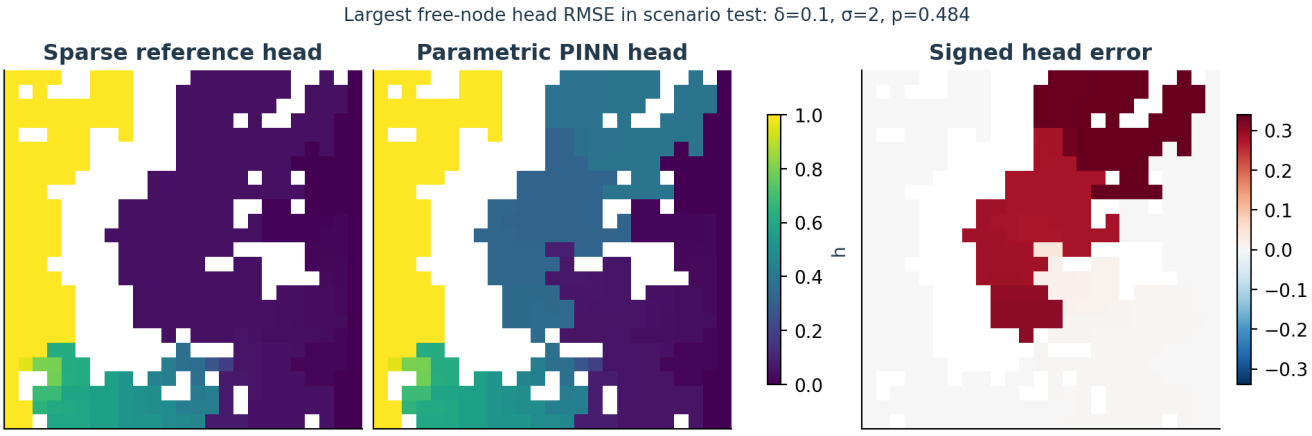


Log-log conductance comparison and cumulative error distributions on positive in-domain cases. Tiny errors are floored only for plotting.

At K=256, training improves mean scenario G error by 2.42× versus the untrained physics network. Much of the final accuracy comes from the graph-physics layers; it cannot be attributed to a small dense encoder alone. Across continuous held-out cases, accurate-mode mean relative G error was 0.0008%, with maximum 0.1448%.

For scenario tests in accurate mode, mean free-node head RMSE is 0.001460; 95th-percentile inlet-flux relative error is 1.1201% and 95th-percentile relative inlet/outlet imbalance is 3.0237%. G derived from energy is more accurate than directly using the approximate boundary flux. Residuals and all raw fluxes are retained; conservation is approximate.

4 · Spatial predictions and stochastic statistics



Deliberately difficult in-domain example: largest free-node head RMSE in the held-out scenario set. Floating nodes are blank; local head errors can persist near weakly connected branches.

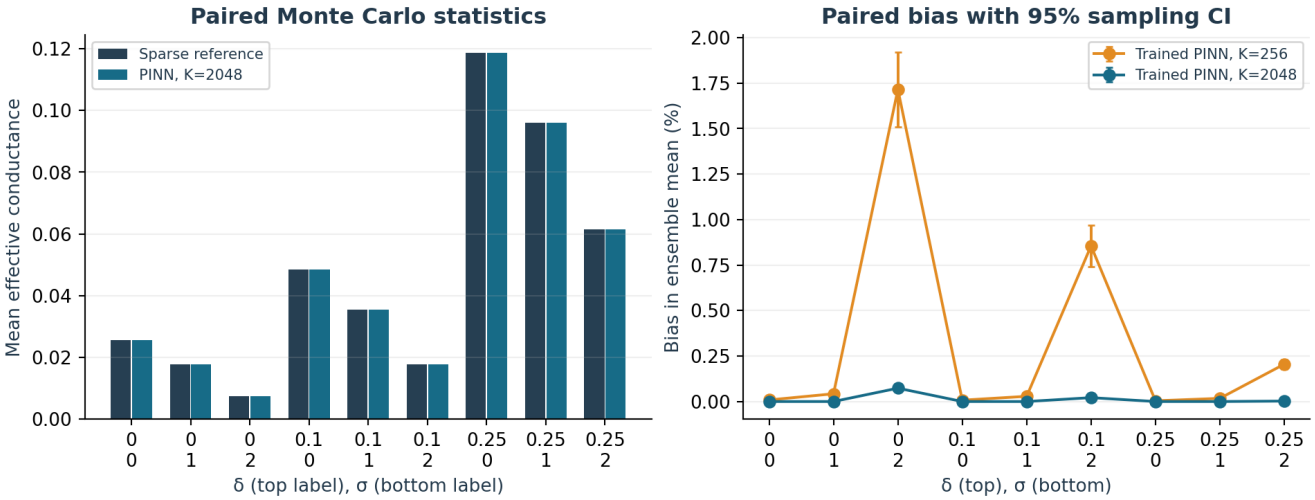
The displayed case has free-node RMSE 0.19366. Energy-weighted output accuracy does not uniformly bound unweighted local head errors in ill-conditioned or weakly connected subgraphs.

δ	σ	Old MC mean[1]	Paired direct mean	PINN mean	Mean bias	P(G=0)
0.00	0	0.02620	0.02555	0.02555	-0.00000%	0.494
0.00	1	0.01825	0.01770	0.01770	0.00000%	0.494
0.00	2	0.00770	0.00733	0.00733	0.07336%	0.494
0.10	0	0.04991	0.04837	0.04837	-0.00000%	0.502
0.10	1	0.03681	0.03549	0.03549	0.00000%	0.502
0.10	2	0.01833	0.01761	0.01761	0.02126%	0.502
0.25	0	0.12216	0.11864	0.11864	-0.00000%	0.498
0.25	1	0.09923	0.09591	0.09591	0.00000%	0.498
0.25	2	0.06385	0.06147	0.06148	0.00246%	0.498

[1]The original study used 10,000 samples per scenario; this independent paired comparison uses 1,000 per scenario. Differences between old and new reference means are Monte Carlo variation. The model discrepancy is the difference within each paired row, not the old-to-new difference.

Means, standard deviations, medians, 95th percentiles, zero probabilities and paired-bias 95% intervals are saved for every scenario and method. Zero probabilities agree exactly by component preprocessing. The bias interval quantifies finite test-sample uncertainty in the mean discrepancy; it is neither a field prediction interval nor a training-uncertainty interval. The ensemble API still samples random fields, but performs no sparse solve for them.

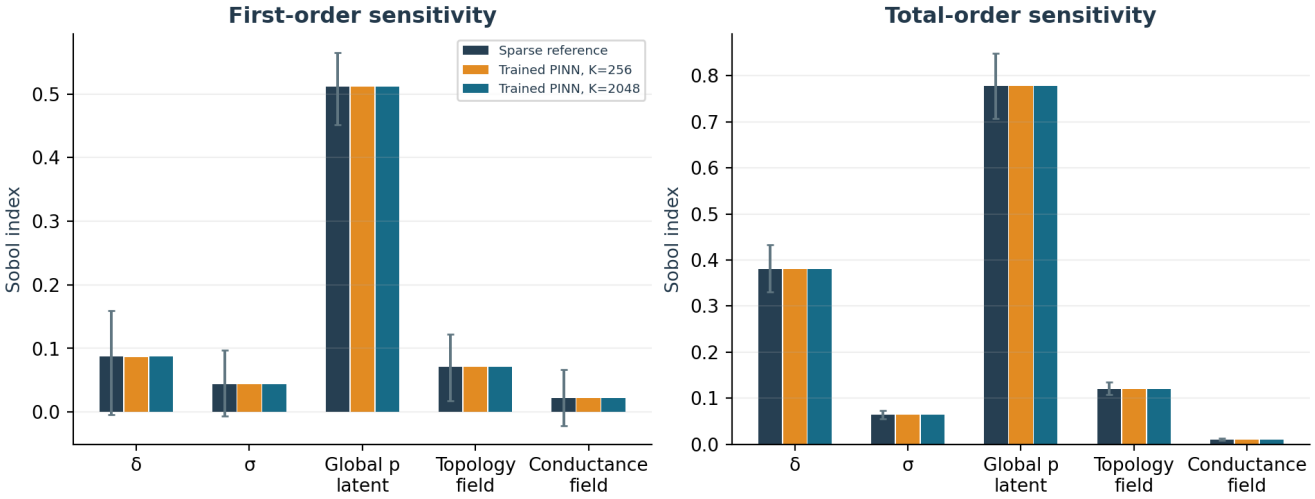
5 · Uncertainty and sensitivity comparison



Same-field Monte Carlo ensemble means and relative biases. Error bars use paired sample differences and normal 95% intervals.

Global sensitivity repeats the grouped Jansen design on 2,048 independent base pairs (14,336 reference fields), with 1,000 paired-row bootstrap resamples. The five independent groups remain δ,σ,U,W,Z. First-order and total indices, their confidence intervals, and paired model-minus-reference differences are all saved.

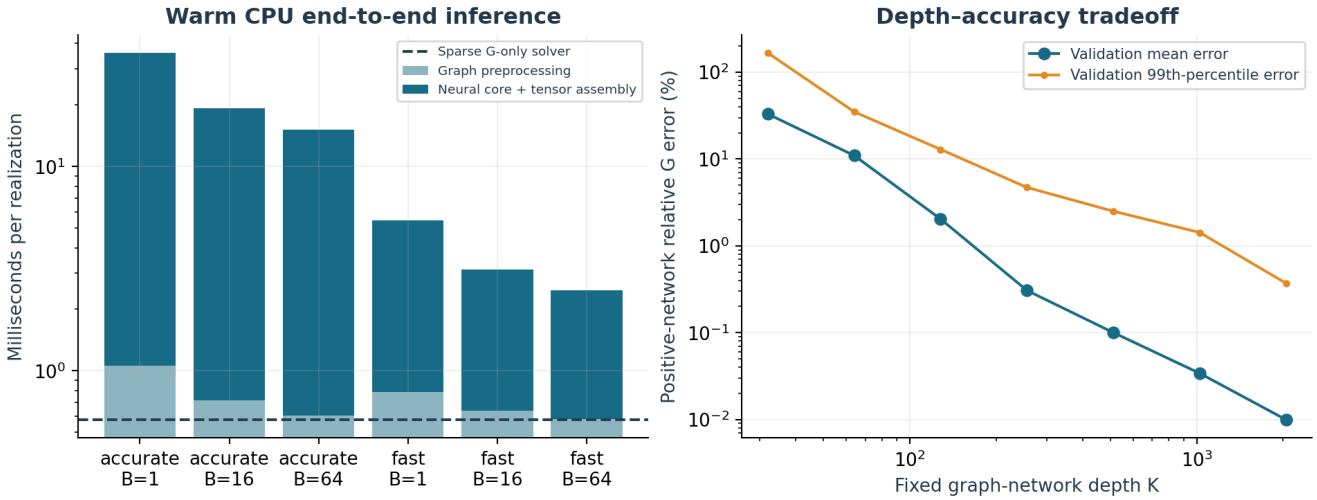
Input group	Direct total index	PINN total index	Paired difference [95% interval]
δ	0.3804	0.3804	1.72e-06 [-1.23e-07, 5.03e-06]
σ	0.0637	0.0637	-3.82e-06 [-7.65e-06, -1.22e-06]
Global p latent U	0.7784	0.7784	2.23e-06 [6.10e-07, 4.96e-06]
Topology field W	0.1204	0.1204	-1.25e-06 [-4.08e-06, 3.11e-07]
Conductance field Z	0.0105	0.0105	-1.17e-06 [-3.30e-06, -1.74e-08]



First and total sensitivity of energy-derived G. Sparse-reference bootstrap intervals are shown; full paired intervals for all methods are in CSV.

This comparison uses a new sensitivity sample, whereas the original study used 16,384 base pairs. Do not compare these point estimates as if their only difference were model error. The paired differences in this table isolate the approximation on identical fields. Rankings depend on priors and latent parameterization; total indices overlap. Topology controls path existence exactly, so σ and Z have no effect on the binary spanning event in either pipeline.

6 · Measured computational cost



Warm CPU inference including graph preprocessing, tensor assembly and neural evaluation. Depth selection uses validation only. The offline training cost is additional.

Mode	Batch	Preprocess ms/case	Core ms/case	Total ms/case	Direct/PINN speedup
Trained, K=2048	1	1.055	34.887	35.942	0.016×
Trained, K=2048	16	0.714	18.619	19.333	0.031×
Trained, K=2048	64	0.606	14.543	15.097	0.040×
Trained, K=256	1	0.786	4.682	5.472	0.105×
Trained, K=256	16	0.639	2.501	3.132	0.178×
Trained, K=256	64	0.579	1.901	2.508	0.226×

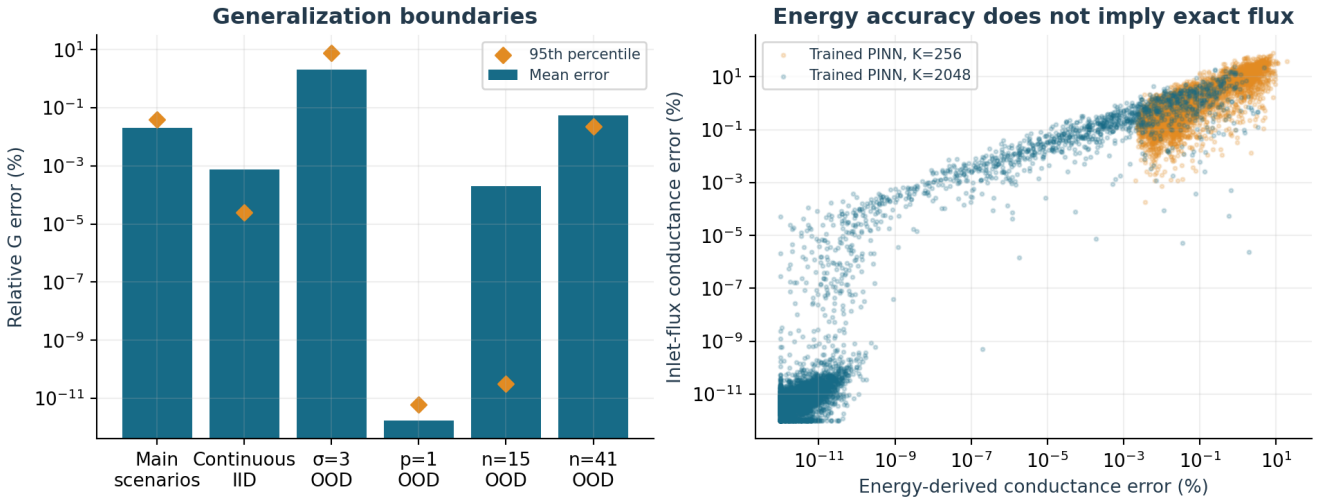
Values are medians of three repetitions on the same 64 new fields. Direct G-only latency was 0.576 ms/case; the CSV also includes full-head sparse timings. The PINN uses two CPU threads, float64 inference and warm loaded weights. Preprocessing includes components and resistance Dijkstra distances; graph generation is excluded for both methods. No GPU results or large-network speedups are inferred.

> The fastest measured PINN setting is 4.35× the sparse G-only latency; no positive break-even is observed. Reusing weights avoids retraining and a sparse solve, but does not imply a runtime advantage. The original sparse method is already inexpensive; this neural architecture’s many fixed graph layers and preprocessing dominate cost.

The high-accuracy mode gives a smaller approximation error at greater inference cost. A positive break-even requires per-input neural cost lower than sparse cost; the measured break-even formula is T_offline/(t_sparse−t_neural) only when that denominator is positive. No training cost is hidden inside the reference timing. Model loading is excluded from warm inference and should be added for cold one-shot use.

7 · Generalization and limits

Test family	Cases	Positive	Mean rel. G error	95th pct.	Maximum
continuous	1000	508	0.0008%	0.0000%	0.1448%
ood_sigma3	250	164	2.0779%	7.6191%	20.2555%
ood_fully_connected	250	250	0.0000%	0.0000%	0.0000%
ood_size15	250	132	0.0002%	0.0000%	0.0251%
ood_size41	250	126	0.0549%	0.0232%	2.5557%



Out-of-domain tests and the distinction between energy and inlet-flux estimates. All comparisons use new physical fields; no test-time correction is applied.

The trained domain is n=25, δ∈[0,0.25], p generated around 0.5, and σ∈[0,2]. The OOD tests use σ=3, fully connected p=1, and n=15 or 41. For other sizes, the same learned weights are reused with regenerated graph-index buffers; no retraining is done. This is exploratory transfer, not a claim of uniform size or parameter generalization.

At σ=3, accurate-mode mean relative G error rises to 2.0779% and the maximum to 20.2555%. Strong contrast and weakly anchored clusters can converge slowly at fixed depth. The dense fully connected case can be easier despite lying outside the training distribution. Small G errors do not certify all heads or fluxes.

Exact boundaries and zero-flow routing are engineered physical constraints, not learned accomplishments. Component analysis and Dijkstra remain required per field. No corrected sparse solve is applied after the model. This architecture does not satisfy a stronger claim of instant scalar-only prediction for every possible random network. Model and sampling error must remain distinct.

8 · Use the trained model and reproduce the study

All new artifacts are separated under `parametric_pinn/`; the original solver, data and reports remain available. The repository contains trained and untrained checkpoints, training histories, complete paired prediction CSVs, sensitivity outputs, timing records, physical diagnostics, spatial examples, an executed notebook, report-generating scripts and editable presentation.

```
python parametric_pinn/scripts/predict.py --delta .25 --sigma 1 --p .6 --seed 42
python parametric_pinn/scripts/predict.py --delta .25 --sigma 1 --draws 1000
python parametric_pinn/scripts/predict.py --conductances field.npy --p .6 --sigma 1
make -C parametric_pinn all
```

The default CLI never calls the reference solver. Use `--compare-reference` explicitly for a check; `--save-heads` exports heads. Physical boundary heads and uniform conductance scale are supported. A .npy field has 1,200 entries, horizontal-then-vertical, row-major, as in the direct solver. API batch prediction reuses one loaded checkpoint. Different seeds can produce different results with the same δ,p,σ; with only distribution parameters, use multiple draws.

Verification and integrity

The 24 original tests plus 12 new PINN tests cover analytic grids, variational bounds, energy gradients, exact boundaries, zero flow, scale equivariance, batch equivalence, no-label training generation, and proof by monkeypatching that prediction does not call the sparse solver. A separate audit rechecks statistics, the zero-flow rule, field-energy consistency, checkpoint identity and the executed notebook. File checksums are in the root inventory.

Scientific conclusion

The study demonstrates a reusable physics-informed parametric graph map with accurate in-domain effective conductance for most cases. The direct solver gives stronger numerical precision and better measured CPU cost for this small lattice. Future work could remove dangling components analytically, learn multilevel preconditioners, optimize batched kernels or train neural operators for larger 3D fracture networks. Such changes require new validation and fair cost accounting, rather than assuming that a PINN is faster or uniformly accurate.

References

[1] Raissi, M., Perdikaris, P., Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Computational Physics* 378, 686–707. doi:10.1016/j.jcp.2018.10.045.

[2] E, W., Yu, B. (2018). The Deep Ritz Method: A Deep Learning-Based Numerical Algorithm for Solving Variational Problems. *Communications in Mathematics and Statistics* 6, 1–12. doi:10.1007/s40304-018-0127-z.

[3] Jansen, M. J. W. (1999). Analysis of variance designs for model output. *Computer Physics Communications* 117, 35–43. doi:10.1016/S0010-4655(98)00154-4.

[4] The supplied mini-project brief and the separately preserved direct-solver study in this repository. The present unrolled graph architecture is a custom discrete, parametric physics-informed model; it is not the original continuum architecture in [1].